

Twenty Most Asked HR Questions for Senior Developers

Customized for: Oleh Pliuta — Senior Angular Developer

Based on: Latest CV — 10+ years, Angular (v1–21), RxJS, NgRx, Ionic/Capacitor, 30+ projects, 8 apps on App Store & Google Play

Career arc: Trident Software → Ergonized (5 yrs) → Shipment Company → iFit (most recent)

Customize: Replace `[bracketed placeholders]` with company-specific details before each interview.

Table of Contents

1. [Question 1: Tell me about yourself and your development journey](#)
2. [Question 2: What interests you about this senior developer position?](#)

3. **Question 3: Describe your leadership style and experience mentoring junior developers**
4. **Question 4: How do you handle conflicts within a development team?**
5. **Question 5: What is your approach to code reviews?**
6. **Question 6: How do you stay current with new technologies and industry trends?**
7. **Question 7: Describe a challenging project you led and how you overcame obstacles**
8. **Question 8: How do you prioritize tasks when working on multiple projects?**
9. **Question 9: What is your experience with agile methodologies?**
10. **Question 10: How do you ensure code quality and maintainability?**
11. **Question 11: Describe a time when you had to make a difficult technical decision**
12. **Question 12: How do you handle tight deadlines and pressure?**
13. **Question 13: What is your approach to technical debt?**

14. [Question 14: How do you communicate technical concepts to non-technical stakeholders?](#)
 15. [Question 15: Describe your experience with system design and architecture](#)
 16. [Question 16: What is your greatest strength as a senior developer?](#)
 17. [Question 17: What areas are you looking to improve or develop further?](#)
 18. [Question 18: How do you handle disagreements with management or product owners?](#)
 19. [Question 19: Where do you see yourself in 3-5 years?](#)
 20. [Question 20: Do you have any questions for us?](#)
-

Questions and Answers

Question 1: Tell me about yourself and your development journey

Purpose: This question helps the interviewer understand your background, experience progression, and passion for development.

How to Answer:

- Start with a brief overview of your years of experience
- Highlight key transitions and growth in your career
- Mention significant achievements or projects
- Connect your journey to the role you're applying for
- Keep it concise (2-3 minutes)

Personalized Answer (Oleh Pliuta):

"I'm a Senior Angular Developer with **10+ years** of experience and **30+ delivered web and mobile applications** across fintech, logistics, healthcare, and e-commerce. I specialize in **Angular, TypeScript, RxJS, NgRx, and Ionic/Capacitor**, with **8 applications published** to the App Store and Google Play.

I started at **Trident Software** building JavaScript web and mobile apps, then spent **five years at Ergonized** as Senior Front-End Developer — B2B insurance quoting with Stripe and NgRx, supply chain dashboards with WebSockets, and warehouse mobile apps with Ionic.

At **Shipment Company**, I built an enterprise vessel logistics platform for **1000+ daily users** — Nx monorepo, shared component libraries, NestJS API integration, and a **14% load-time improvement** through webpack tuning and lazy loading. I

also led an **Angular 14 → 17 migration** across 80+ components in six weeks without disrupting active feature work.

Most recently at **iFit**, I worked on fitness web and mobile apps serving **1M+ registered users** with Angular 19, Ionic 7, and React 19. I migrated core modules to **standalone component architecture**, cut the initial bundle from **3.2MB to 2.1MB**, improved the Lighthouse score from **61 to 84**, grew test coverage on new modules from **0% to 70%+** with Jest, and shipped **3 Ionic/Capacitor releases** to the App Store and Google Play.

What excites me about this opportunity is **[specific technology/challenge from the job description]** — my experience in **[relevant skill: e.g., Angular architecture, shared libraries, performance, mentoring]** aligns well with your team's needs."

Key Points to Remember:

- Be authentic and enthusiastic
- Focus on relevant experience
- Show progression and growth
- Demonstrate self-awareness
- Connect your story to the company's needs

[↑ Back to Table of Contents](#)

Question 2: What interests you about this senior developer position?

Purpose: Assesses your motivation, research about the company, and alignment with the role.

How to Answer:

- Show you've researched the company and role
- Mention specific technologies or challenges that excite you
- Align your interests with company goals
- Demonstrate enthusiasm and genuine interest
- Avoid generic responses

Personalized Answer (Oleh Pliuta):

"I'm particularly excited about this position for several reasons:

First, the tech stack and senior scope. The role emphasizes **Angular architecture, shared libraries, performance, and mentoring** — not just feature delivery. That's exactly what I've been doing: Nx monorepos at Shipment Company, standalone

migrations and bundle optimization at iFit, and testing standards at Ergonized.

Second, the technical challenges. I've solved similar problems at scale — reducing initial bundle size by **34%** at iFit, cutting load times by **14%** at Shipment Company, and building real-time dashboards with WebSockets at Ergonized. The opportunity to work on **[specific product/challenge from job description]** at **[company]** is compelling.

Third, your engineering culture. I researched **[company initiative/blog/values]** and it resonates with how I work — **[e.g., design thinking, quality craft, experimentation with discipline, collaborative reviews]**.

Finally, the leadership dimension. I want to keep growing as a senior IC — shaping standards, mentoring developers, and making architectural decisions that last. This role's emphasis on **[mentoring/standards/architecture from posting]** aligns with where I want to go next."

What NOT to Say:

- "I just need a job" or "The salary is good"
- Generic responses that could apply to any company

- Only focusing on what you'll gain, not what you'll contribute
- Negative comments about your current employer

[!\[\]\(34b4f260a8587d2e97eeaee361cc357b_img.jpg\) Back to Table of Contents](#)

Question 3: Describe your leadership style and experience mentoring junior developers

Purpose: Evaluates your ability to lead, mentor, and develop team members—crucial for senior roles.

How to Answer:

- Describe your specific leadership approach
- Provide concrete examples of mentoring
- Show results and impact
- Demonstrate empathy and patience
- Highlight different mentoring scenarios

Personalized Answer (Oleh Pliuta):

"My leadership style is collaborative and coaching-oriented — I lead by example while empowering team members to take ownership.

Mentoring Approach:

- 1. Pair programming on real problems:** At Shipment Company, a mid-level developer struggled with **RxJS memory leaks** on long-lived operational screens. I paired with them on `takeUntilDestroyed` patterns, added ESLint guidance for subscriptions, and they became the team's reviewer for streaming code.
- 2. Code reviews as teaching:** I explain the *why* behind suggestions — not just "change this." At Ergonized, detailed educational reviews were part of a culture that contributed to **~20% fewer production bugs**.
- 3. Establishing standards, not just fixing bugs:** I introduced frontend **unit testing standards** at Ergonized (Jasmine/Karma, coverage growing from near-zero to **55%** on core modules) and **Playwright E2E** for critical user flows. At iFit, I grew test coverage on new Angular modules from **0% to 70%+** with Jest.
- 4. Progressive responsibility:** I start mentees on isolated features, then full modules, then architectural decisions — matching complexity to their growth pace.

5. RFCs and lunch-and-learns: When shared grid wrappers diverged across squads at Shipment Company, I wrote an RFC, refactored the wrapper, added tests and docs, and ran sessions — reaching **~80% adoption** within a quarter.

Results: Stronger review culture, fewer recurring bug classes, and developers who can own features independently faster."

Key Leadership Qualities to Highlight:

- Empathy and emotional intelligence
- Clear communication
- Patience and adaptability
- Focus on growth and development
- Results-oriented mentoring

[!\[\]\(339a16584d5da0f0a3ca4e9ec17bf6a1_img.jpg\) Back to Table of Contents](#)

Question 4: How do you handle conflicts within a development team?

Purpose: Tests your conflict resolution skills, emotional intelligence, and team management abilities.

How to Answer:

- Acknowledge that conflicts are normal and can be productive
- Provide a specific example with context, action, and resolution
- Show empathy and fairness
- Demonstrate communication skills
- Focus on positive outcomes

Personalized Answer (Oleh Pliuta):

"I view conflicts as opportunities for growth when handled constructively. My approach: address early, listen to all sides, then decide with data — not opinions.

Real Example:

At Shipment Company, a colleague wanted to introduce a **new state management library** mid-project while the team was already productive with **RxJS + services**. Tension was affecting sprint planning.

My Approach:

- Met with them individually to understand concerns — valid points on both sides
- Proposed a **time-boxed two-day spike** with clear criteria: bundle size, test complexity, team familiarity, migration cost

- Both options were evaluated against our actual requirements — enterprise ops UI, not a greenfield app
- **RxJS + services won** on pragmatism; we documented the decision in an **ADR** and moved on

Outcome: No lingering resentment, one pattern for the team, and a reusable decision process for future disagreements. The key was assuming good intent and replacing opinion battles with measurable spikes.

Conflict Resolution Principles:

- Stay neutral and objective
- Focus on issues, not personalities
- Create win-win solutions when possible
- Follow up to ensure resolution
- Learn from each conflict to prevent future issues

[!\[\]\(23d9fc146e83b5c3013cfa32c784f8d5_img.jpg\) Back to Table of Contents](#)

Question 5: What is your approach to code reviews?

Purpose: Assesses your understanding of code quality, collaboration, and knowledge sharing.

How to Answer:

- Explain your systematic approach
- Balance thoroughness with pragmatism
- Emphasize learning and growth, not criticism
- Mention specific aspects you review
- Show respect for developers' work

Personalized Answer (Oleh Pliuta):

"I view code reviews as one of the most valuable practices for knowledge sharing and quality. My approach is structured yet respectful:

What I Review:

- Functionality, edge cases, and RxJS subscription cleanup
- Alignment with team patterns (Nx library boundaries, OnPush where appropriate)
- Test coverage — especially on shared library code where one bug affects every consumer
- Performance implications (bundle impact, unnecessary re-renders)

Communication Style:

- Ask questions: *"What if we used takeUntilDestroyed here?"* instead of demanding changes
- Categorize feedback:  blocking (bugs, security) ·  important (maintainability) ·  nit (style)
- Always acknowledge good solutions

Real Example:

At Shipment Company, a developer implemented a complex caching layer for a data grid. It worked but was hard to test. I praised the performance thinking, shared maintainability concerns, and **pair-programmed a simpler approach** with clear trade-offs documented. The developer learned a durable pattern, and we shipped cleaner code.

Time Management: Small PRs get 15–30 minutes; large PRs get split reviews or a suggestion to break the PR down. I don't let reviews block the team."

[!\[\]\(950a62bbddad88d64435fd35607dfc42_img.jpg\) Back to Table of Contents](#)

Question 6: How do you stay current with new technologies and industry trends?

Purpose: Demonstrates your commitment to continuous learning and adaptability in a rapidly changing field.

How to Answer:

- Show a systematic approach to learning
- Mention specific resources and activities
- Balance breadth and depth
- Demonstrate practical application, not just theoretical knowledge
- Show selectivity—you can't learn everything

Personalized Answer (Oleh Pliuta):

"Continuous learning is essential in frontend — especially with Angular's rapid evolution. I stay current with a practical, selective approach:

Daily / Weekly:

- **Angular Weekly**, **JavaScript Weekly**, and official Angular blog/changelog for signals, standalone components, and new APIs
- Review documentation when upgrading — recently deep-dived **Angular Signals** and **standalone component migration** during the iFit refactor
- Friday team knowledge sharing when available

Applied Learning (recent examples):

- **iFit:** Migrated core modules to standalone architecture, applied lazy loading across 12 feature modules, integrated **LLM-based API** with streaming UI in React 19
- **Shipment Company:** Led **Angular 14 → 17 migration** across 80+ components; evaluated **Module Federation** patterns for micro-frontend readiness
- **Ergonized:** Introduced **Playwright E2E** and unit testing standards from near-zero to 55% coverage

How I Filter What to Learn:

1. **Relevance** — Will this help my current or next project?
2. **Adoption** — Is the industry moving here? (standalone components, signals, Nx monorepos)
3. **Fundamentals** — Does it deepen core understanding? (change detection, RxJS vs signals)
4. **Interest** — Am I genuinely curious?

What I Don't Do: Chase every new library, adopt tools without a spike, or abandon fundamentals for hype.

I learn by **applying** — spikes, migrations, team presentations — not just consuming content."

[↑ Back to Table of Contents](#)

Question 7: Describe a challenging project you led and how you overcame obstacles

Purpose: Evaluates problem-solving skills, leadership under pressure, and ability to deliver results.

How to Answer:

- Use the STAR method (Situation, Task, Action, Result)
- Choose a genuinely challenging project with measurable outcomes
- Highlight both technical and leadership aspects
- Be honest about difficulties
- Show what you learned

Personalized Answer (Oleh Pliuta):

Situation:

At **iFit**, the fitness platform serves **1M+ registered users** across web and mobile. The Angular app had grown organically — large initial bundle (**3.2MB**), slow dashboard load (**Lighthouse performance score of 61**), and **0% test coverage** on newer

modules. Feature development couldn't pause; users open the app daily for workouts.

Task:

Improve performance and maintainability while continuing to ship features — including AI-powered workout recommendations and mobile releases.

Challenges:

- Migrating to **standalone component architecture** without breaking existing NgModule-based code
- Coordinating **web (Angular 19) and mobile (Ionic 7/Capacitor)** releases with iOS/Android teams
- Balancing performance work with AI feature delivery in **React 19**

Actions:

1. **Incremental migration:** Moved core modules to standalone components module-by-module, reducing bootstrap time by **18%**
2. **Lazy loading:** Introduced lazy loading across **12 feature modules** — initial bundle **3.2MB** → **2.1MB (34% reduction)**

3. **Performance pass:** OnPush change detection, bundle analysis, Lighthouse-driven fixes — score **61 → 84**
4. **Testing foundation:** Grew unit test coverage on new modules from **0% to 70%+** with Jest
5. **Cross-team coordination:** Shipped **3 App Store / Google Play releases** while performance work ran in parallel

Critical Obstacle — Stale grid data:

A performance optimization on a live data grid caused **stale status display** under burst updates in staging. I rolled back via feature flag, root-caused the batching issue, and added a regression test — **no production impact**.

Results:

- **34% smaller initial bundle, 18% faster bootstrap,**
Lighthouse **61 → 84**
- **70%+ test coverage** on new modules
- **3 successful mobile releases** during the optimization period
- Team gained confidence in incremental Angular migrations

Key Learning: Large Angular improvements succeed through **incremental, measurable steps** — not big-bang rewrites — with feature flags and regression tests as safety nets."

[↑ Back to Table of Contents](#)

Question 8: How do you prioritize tasks when working on multiple projects?

Purpose: Assesses time management, decision-making skills, and ability to handle competing priorities.

How to Answer:

- Describe your prioritization framework
- Show how you balance technical and business needs
- Demonstrate communication about priorities
- Mention tools and techniques
- Show flexibility and adaptability

Personalized Answer (Oleh Pliuta):

"As a senior developer, I juggle feature delivery, architecture work, code reviews, and mentoring. I prioritize by **user impact and risk**:

My Framework:

1. **Must be correct** — auth, payments, data integrity (e.g., Stripe flows on Ergonized insurance platform)

2. **Must ship** — core user path; negotiate scope with PM early

3. **Nice to have** — polish, secondary views

I also use **Eisenhower Matrix** thinking: production bugs and blocking issues first, then high-impact architecture, then tech debt.

Real Example — Logistics module mid-sprint:

Delivering a new module at Shipment Company with NestJS backend, QA, PM, and mobile team — **API contracts changed mid-sprint.**

What I Did:

- Fixed any production blockers immediately
- Daily 15-min sync with backend; same-day notes on API changes
- Sliced frontend into shippable increments
- Flagged risk early with **scope vs timeline options** for PM
- Communicated **green/yellow/red** status daily

Result: Shipped on time, zero P1 bugs.

Communication: I give realistic estimates, communicate delays early, and escalate when multiple high-priority items conflict.

Surprises are worse than bad news delivered on time.

Tools: JIRA for sprint tasks, calendar blocking for deep work, Nx affected commands to focus CI on what changed."

[↑ Back to Table of Contents](#)

Question 9: What is your experience with agile methodologies?

Purpose: Evaluates understanding of agile practices and ability to work in iterative, collaborative environments.

How to Answer:

- Describe specific agile frameworks you've used
- Provide concrete examples of implementation
- Show understanding of agile principles, not just ceremonies
- Discuss adaptations based on team needs
- Mention both successes and challenges

Personalized Answer (Oleh Pliuta):

"I've worked in **Scrum** and **Kanban** environments across Ergonized, Shipment Company, and iFit for **10+ years** — always as a hands-on senior contributor, not just a process follower.

Scrum in Practice (Ergonized, Shipment Company, iFit):

- **2-week sprints** with planning, daily standups, reviews, and retrospectives
- Broke down user stories into technical tasks; provided estimates using planning poker
- Demonstrated features in sprint reviews — e.g., insurance quoting flows, logistics dashboards, fitness app modules
- Used retros to improve process — introduced Playwright E2E at Ergonized after recurring production bugs on critical paths

Agile Engineering Practices I Champion:

- **CI/CD** with lint, unit tests, and bundle budgets on PRs (Nx monorepo at Shipment Company)
- **Incremental delivery** — Angular migrations module-by-module at iFit, not big-bang rewrites
- **Feature flags** for safe rollouts (used during performance optimization rollback)
- **Code reviews** as continuous knowledge sharing

Real Example — API change mid-sprint:

At Shipment Company, backend API contracts changed during a logistics module sprint. Instead of hiding in code, I ran **daily 15-**

min syncs, documented changes same-day, and sliced frontend into shippable increments. Shipped on time with zero P1 bugs.

My Philosophy: Agile is about **delivering value incrementally** and **responding to change** — not ceremony theater. If a meeting doesn't help the team ship better software, I question it. I prefer Scrum's structure for feature teams and Kanban flexibility for platform work."

[↑ Back to Table of Contents](#)

Question 10: How do you ensure code quality and maintainability?

Purpose: Assesses your understanding of software craftsmanship and long-term code health.

How to Answer:

- Describe multiple layers of quality assurance
- Balance automated and manual approaches
- Show understanding of trade-offs
- Provide specific practices and tools
- Demonstrate long-term thinking

Personalized Answer (Oleh Pliuta):

"Code quality and maintainability are fundamental — especially in **shared libraries** where one bug affects every team. I use a multi-layered approach:

1. Automated Quality Gates:

- **ESLint + Prettier + TypeScript strict mode** — enforced in CI, not optional
- **Pre-commit hooks** and PR checks that block merges on lint/test failures
- **Bundle budgets** in CI at Shipment Company — prevented performance regressions

2. Testing Strategy (by role):

- **Ergonized:** Established unit testing standards (Jasmine/Karma, **0% → 55%** coverage on core modules); introduced **Playwright E2E** on critical insurance quoting and payment flows
- **iFit:** Grew Jest coverage on new Angular modules from **0% → 70%+**
- **Shipment Company:** Higher coverage requirements on **shared Nx library code** — highest leverage in a monorepo

Testing mix: Unit tests for services/RxJS logic · component tests for forms and error states · selective E2E on business-critical paths · regression tests on every bug fix

3. Modular Architecture:

- **Nx monorepo** with feature / UI / data-access libs and **boundary tags** at Shipment Company
- **Standalone components** migration at iFit — simpler module graph, better tree-shaking
- Shared libraries treated as **products**: clear APIs, tests first, docs where needed

4. Code Reviews:

- Every PR reviewed; shared lib PRs need extra scrutiny
- Reviews teach patterns (RxJS cleanup, OnPush, defensive error states) — not just gatekeep

5. Technical Debt Management:

- Boy Scout Rule on every PR
- **~20% sprint capacity** for improvements when possible
- ADRs for significant choices (state management, testing tools, migration strategy)

Real Impact:

- Ergonized review/refactoring culture contributed to **~20% fewer production bugs**
- iFit: measurable quality gains alongside **Lighthouse 61 → 84** performance work
- Shipment Company: shared grid RFC reached **~80% adoption** — consistency at scale

Philosophy: Quality enables speed in the long run. I never skip tests or reviews on auth, payments, or shared library code — even under deadline pressure."

[↑ Back to Table of Contents](#)

Question 11: Describe a time when you had to make a difficult technical decision

Purpose: Evaluates decision-making process, technical judgment, and ability to balance trade-offs.

How to Answer:

- Describe the context and constraints
- Explain the options you considered

- Show your decision-making criteria
- Discuss the outcome and learnings
- Be honest about uncertainties

Personalized Answer (Oleh Pliuta):

"At **Shipment Company**, I faced a technical decision that would affect every squad on our enterprise logistics platform.

Context:

We were mid-delivery on a new logistics module. A colleague proposed introducing a **new state management library** while the team was already productive with **RxJS + services**. The disagreement was affecting sprint planning — one path meant migration cost and learning curve; the other risked missing modern patterns.

Why It Was Difficult:

- Both sides had valid technical arguments
- We couldn't afford weeks of debate — **1000+ daily users** depended on on-time delivery
- Wrong choice would create **two patterns for the same problem** across the Nx monorepo

My Decision Process:

1. **Defined criteria:** bundle size impact, test complexity, team familiarity, migration cost, time to productivity
2. **Proposed a time-boxed two-day spike** — both options built against the same user story
3. **Measured objectively:** lines of code, test setup effort, developer feedback
4. **Documented in an ADR** regardless of outcome

Decision: Stay with RxJS + services

Rationale:

- Team was already productive; migration cost outweighed benefits for our use case
- Enterprise ops UI — not a greenfield app needing a new paradigm
- Clear path to **Angular Signals** for local state later without a full rewrite

How I Brought the Team Along:

- Shared spike results transparently — both options presented fairly

- Gave the advocate ownership of the ADR and future signals evaluation
- Set a checkpoint to reassess in 6 months

Outcome:

- Team moved on without lingering tension
- One consistent pattern across squads
- Reusable decision process for future disagreements

What I Learned: Difficult decisions need **data and timeboxes**, not opinion battles. Document decisions (ADRs) so future developers understand the *why*."

[!\[\]\(950a62bbddad88d64435fd35607dfc42_img.jpg\) Back to Table of Contents](#)

Question 12: How do you handle tight deadlines and pressure?

Purpose: Assesses stress management, prioritization under pressure, and decision-making when time-constrained.

How to Answer:

- Show you can work under pressure without panic
- Demonstrate strategic thinking

- Explain how you maintain quality
- Provide specific examples
- Show you know your limits

Personalized Answer (Oleh Pliuta):

"Tight deadlines are inevitable — especially with **mobile release cycles** and enterprise delivery. I handle pressure through **ruthless prioritization** and **early communication**, not sustained overtime.

My Approach:

1. MoSCoW prioritization:

- **Must have:** Auth, payments, data integrity (Stripe on Ergonized insurance platform)
- **Must ship:** Core user path — negotiate scope with PM early
- **Could have:** Polish, secondary views — explicitly cut under pressure

2. Communicate green/yellow/red daily when pressure rises. Surprises are worse than bad news delivered early.

3. Never cut: Tests and reviews on auth, payments, or shared library code — even under deadline.

Real Example — iFit mobile release + performance work:

We had parallel pressure: ship **3 Ionic/Capacitor releases** to App Store/Google Play while running a **bundle optimization pass** (3.2MB → 2.1MB) and **standalone component migration**.

What I Did:

- Scoped performance work to **highest-traffic modules first** — dashboard Lighthouse **61 → 84**
- Used **feature flags** so optimization could roll back safely (learned from a staging incident with stale grid data)
- Negotiated with PM: AI workout UI ships on schedule; secondary views deferred one sprint
- Coordinated daily with iOS/Android teams on release blockers

Result: All 3 mobile releases shipped; **34% bundle reduction** and **70%+ test coverage** on new modules delivered without production incidents.

What I Avoid: Sustained overtime, skipping code review, or creating tech debt on critical paths without a follow-up ticket.

Philosophy: Pressure should drive **focus and scope negotiation**, not panic or quality shortcuts."

[↑ Back to Table of Contents](#)

Question 13: What is your approach to technical debt?

Purpose: Evaluates your understanding of long-term code health and ability to balance speed with sustainability.

How to Answer:

- Show you understand what technical debt is
- Explain how you identify and track it
- Demonstrate balanced approach to addressing it
- Provide examples of managing it
- Show business awareness

Personalized Answer (Oleh Pliuta):

"Technical debt is inevitable — I treat it like financial debt: sometimes strategic, but always visible and managed.

Intentional debt I've taken:

- Shipping AI workout UI at iFit before full test coverage — documented follow-up to reach **70%+** on new modules

- Running NgModule and standalone components in parallel during migration — paid down module-by-module

How I manage it:

1. Prevent unnecessary debt: ESLint/Prettier/TypeScript strict in CI; code reviews; ADRs for big choices

2. Make it visible: TODO comments with ticket refs; tech debt items on sprint board; quarterly health discussions

3. Pay incrementally: Boy Scout Rule on every PR; **~20% sprint capacity** for improvements when possible

4. Communicate in business terms:

-  *"We need to refactor the auth module"*
-  *"This Angular 14 codebase adds 2 days to every feature. A 6-week migration to Angular 17 — which I led across 80+ components at Shipment Company — will speed delivery and reduce bugs."*

Real Examples:

- **Shipment Company:** Introduced **Nx monorepo** to consolidate web/mobile codebases — **~30% less duplicated code** across

3 apps; led **Angular 14 → 17 migration** in 6 weeks without halting features

- **Ergonized:** Grew test coverage from near-zero to **55%**; Playwright E2E on critical insurance/Stripe flows — contributed to **~20% fewer production bugs**
- **iFit:** Paid down bundle debt (**3.2MB → 2.1MB**) alongside feature delivery

Red lines — never accept debt on: security vulnerabilities, payment/data integrity, auth flows

Philosophy: Sustainable velocity beats perfect code. Document trade-offs, pay interest regularly, and never let debt become invisible."

[!\[\]\(a03a7eb2f4046e1d3c76772003e549ea_img.jpg\) Back to Table of Contents](#)

Question 14: How do you communicate technical concepts to non-technical stakeholders?

Purpose: Assesses your ability to bridge the gap between technical and business teams.

How to Answer:

- Show empathy for non-technical audience
- Provide specific communication strategies
- Use examples
- Demonstrate business awareness
- Show patience and adaptability

Personalized Answer (Oleh Pliuta):

"Most of my stakeholders are PMs, operations managers, and business owners — not engineers. I translate technical work into **business impact**.

Principles:

1. **Know the audience** — executives want ROI/risk; PMs want timelines/trade-offs
2. **Use analogies** — lazy loading is like *only loading the kitchen equipment you need for today's menu, not the entire restaurant*
3. **Lead with outcomes, not implementation**

Real Examples:

Bundle size / performance (iFit):

- **✗** *"We need lazy loading and standalone components"*

-  *"Users wait too long on first open. We can cut load size by **34%** and improve the performance score from **61 to 84** — that means faster workouts and fewer drop-offs. It takes ~2 sprints alongside feature work."*

Payment flows (Ergonized insurance platform):

-  *"We need idempotent Stripe API calls with server-side reconciliation"*
-  *"If a user clicks Pay twice, we must never charge twice. The UI shows clear loading states; the server is the source of truth — same care I'd apply to wallet flows in regulated products."*

Migration timeline (Shipment Company):

-  *"Angular 14 → 17 across 80 components"*
-  *"Our framework is 3 versions behind, which slows every new feature. A **6-week phased migration** lets us keep shipping while reducing future delivery time and security risk."*

When explaining delays: I give the iceberg analogy — visible UI is ~20%; the rest is testing, edge cases, API contracts, and operational readiness.

I always check: *"Does that make sense? What would you like me to clarify?"*

[↑ Back to Table of Contents](#)

Question 15: Describe your experience with system design and architecture

Purpose: Assesses your ability to design scalable, maintainable systems and think at a high level.

How to Answer:

- Describe specific architectural patterns you've used
- Explain design decisions and trade-offs
- Show understanding of scalability and reliability
- Provide concrete examples
- Demonstrate ability to match architecture to business needs

Personalized Answer (Oleh Pliuta):

"I design **frontend architecture** that matches team size and product complexity — not every app needs micro-frontends on day one.

1. Nx Monorepo — Shipment Company (primary example)

Situation: Enterprise vessel logistics platform — web + Android — **1000+ daily users**. Teams duplicated UI between surfaces.

Design:

- **Nx monorepo** with **feature / UI / data-access libraries**
- **Boundary tags** enforcing dependency rules between libs
- **Adapters** for web vs mobile — same data model, different presentation shells
- **NestJS REST APIs** with typed DTOs and defensive error states
- Lazy routes, buildable libs, **bundle budgets in CI**

Results: ~30% less duplicated code across 3 apps; 14% faster TTI; shared library adopted across squads

2. Modular Angular + State — Ergonized

- **NgRx** for complex B2B insurance quoting flows (multi-step wizard, dynamic underwriting rules)
- **Stripe** payment integration with idempotent UI states and E2E tests on critical paths
- Supply chain dashboard: **WebSockets** for live map updates with throttled/batched rendering

3. Performance & Scale — iFit

- **Standalone component** migration reducing bootstrap time **18%**
- Lazy loading across **12 feature modules** — bundle **3.2MB** → **2.1MB**
- Platform serving **1M+ users** — OnPush, virtual scroll, realistic-data performance testing

4. Multi-surface / Multi-brand patterns

- Web + mobile from shared libs (Shipment Company, iFit Ionic/Capacitor)
- Same pattern applies to **multi-brand theming** — one codebase, multiple operator skins

Design Principles:

- **Match complexity to need** — monorepo before micro-frontends unless teams truly deploy independently
- **Clear contracts** between UI, state, and API layers
- **Measure with real data** — not empty-state Lighthouse
- **ADRs** for significant choices (state lib, migration strategy, library boundaries)

Aware of (not deepest experience): Module Federation / micro-frontends — same boundary discipline; eager to apply where teams deploy independently

Philosophy: Good architecture solves business problems with the **simplest structure that scales** — shared libraries, typed APIs, testable boundaries, and performance budgets users actually feel."

[↑ Back to Table of Contents](#)

Question 16: What is your greatest strength as a senior developer?

Purpose: Highlights your unique value proposition and self-awareness.

How to Answer:

- Choose 1-2 strengths with concrete evidence
- Connect to the role requirements
- Be confident but not arrogant
- Provide measurable examples

Personalized Answer (Oleh Pliuta):

"My greatest strength is **platform thinking** — I don't just ship features; I build **shared libraries, modular architecture, and standards** that make the whole team faster.

Evidence:

- **Shipment Company:** Nx monorepo with feature/UI/data-access libs — **~30% less duplicated code, 14% faster TTI**, shared grid RFC with **~80% squad adoption**
- **Ergonized:** Testing standards and review culture — **~20% fewer production bugs**
- **iFit:** Standalone migration + lazy loading — **34% bundle reduction**, Lighthouse **61 → 84**

Secondary strength: Calm delivery under complexity — enterprise B2B flows (insurance quoting, logistics ops), real-time WebSocket dashboards, and high-stakes Stripe payments. I debug cross-stack issues because I understand both **Angular frontends and NestJS APIs**.

These map directly to senior frontend roles: architecture, REST integration, performance, and raising the team's baseline."

Question 17: What areas are you looking to improve or develop further?

Purpose: Tests self-awareness, growth mindset, and honesty.

How to Answer:

- Choose a real area — not a humble brag
- Show what you're doing to improve
- Keep it relevant to senior level
- Don't undermine your candidacy

Personalized Answer (Oleh Pliuta):

"I sometimes **invest too long in root-cause analysis** when a pragmatic workaround would unblock the team faster — especially on tricky RxJS or performance issues where I want to understand *why*, not just *what*.

How I'm improving:

- **Timeboxing investigations** — e.g., 2 hours max before presenting options to the team
- **Communicating trade-offs early** — 'Here are three paths: quick fix now, proper fix next sprint, or spike to decide'
- **Delegating follow-up** when the immediate unblock matters more than the deep dive

I've gotten better at this — the grid performance rollback at Shipment Company was caught quickly because I had feature flags and a regression test mindset, not because I spent days in isolation."

Question 18: How do you handle disagreements with management or product owners?

Purpose: Evaluates diplomacy, communication, and ability to influence without authority.

How to Answer:

- Show respect for business priorities
- Use data and options, not emotion
- Describe a specific example
- Demonstrate willingness to commit once decided

Personalized Answer (Oleh Pliuta):

"I assume good intent and lead with **data and options**, not blocking or passive-agreement.

Example — Scope vs timeline:

PM wanted a full logistics module plus performance overhaul in

one sprint. I didn't say no — I presented three options with trade-offs:

Option	Scope	Timeline	Risk
A	Full module + perf	3 weeks	Quality risk on payments path
B	Core module now, perf next sprint	2 weeks	Acceptable — my recommendation
C	MVP module only	1.5 weeks	Missing reporting views

We agreed on **Option B**. I committed fully once decided — no revisiting the decision in standups.

Example — Technical disagreement:

When a colleague wanted a new state library mid-project, I proposed a **time-boxed spike** with criteria (bundle size, test complexity, team familiarity). We documented the outcome in an **ADR** and moved on.

Principles: Disagree constructively in private/design sessions; **disagree and commit** in public; escalate only when risk is

genuinely high (data integrity, security, auth)."

Question 19: Where do you see yourself in 3-5 years?

Purpose: Assesses ambition, retention potential, and alignment with company growth paths.

How to Answer:

- Align with realistic paths at the company (staff IC vs tech lead)
- Show continued hands-on technical depth
- Mention mentoring and broader impact
- Avoid titles that don't exist at the company

Personalized Answer (Oleh Pliuta):

"In **3–5 years**, I see myself still building **high-impact frontend systems** — either as a **staff-level IC** shaping architecture, shared libraries, and performance standards, or as a **tech lead** balancing hands-on coding with mentoring.

I want depth in **product-minded Angular engineering** — the kind of work I've done at iFit, Shipment Company, and Ergonized — and to be someone who makes the **team and codebase** better, not just ships tickets.

Concretely: leading larger **Nx/monorepo** initiatives, mentoring mid-level developers into seniors, and contributing to **frontend standards** that outlast any single feature. I'm not looking to move into pure management — I want to stay close to the code while multiplying impact through the team."

Question 20: Do you have any questions for us?

Purpose: Shows preparation, genuine interest, and senior-level thinking. "No questions" is a red flag.

How to Answer:

- Prepare 4–5 questions before every interview
- Mix role, team, tech, and culture
- Ask questions you actually care about
- Avoid questions easily answered by the website

Personalized Answer (Oleh Pliuta) — pick 3–4 per interview:

1. *"What product or client project would I join first — greenfield Angular app or evolving an existing platform?"*
2. *"How is the frontend team structured — headcount, lead, relationship to backend squads?"*

3. *"Does the team use a **monorepo**, **micro-frontends**, or a mix — and how mature is the shared library layer?"*
4. *"What does success look like in the first **6 months** for this senior role?"*
5. *"What are the **next interview steps** after this call?"*
6. *"How does the team balance **new features vs technical debt** — is there explicit sprint capacity for platform work?"*
7. *"What's the biggest **frontend technical challenge** the team faces that you'd hope a senior hire could help solve?"*
8. *"How does the team approach **testing and CI/CD** on the frontend — unit, component, E2E, bundle budgets?"*

Tip: End with *"Based on what we discussed, is there anything about my background you'd like me to clarify?"* — shows confidence and closes gaps.

[!\[\]\(3d8c13c92b853674f749aac6fa869926_img.jpg\) Back to Table of Contents](#)